

# **INTERNSPARK MERN STACK INTERNSHIP**

## **TASK 3: Authentication & Protected Routes (JWT)**

**Project Name:** InternHub – Internship Management Portal

**Submitted By:** MD Sohail Ahmed

### **Objective:**

The objective of this task was to implement a secure authentication system using JWT (JSON Web Tokens), bcrypt password hashing, protected API routes, and protected React pages.

### **Technologies Used:**

Backend:

- Node.js
- Express.js
- MongoDB
- Mongoose
- JWT
- bcryptjs
- cookie-parser

Frontend:

- React.js
- React Router DOM
- Axios
- Tailwind CSS

### **Implementation Steps:**

1. User Authentication Model  
A User model was created to store user information such as name, email, and hashed password.
2. Password Hashing  
Passwords were securely hashed using bcrypt before storing them in MongoDB.
3. User Registration API  
A registration endpoint was implemented to allow new users to create accounts.

Endpoint:

POST /api/auth/register

4. User Login API

A login endpoint was implemented to verify user credentials and generate a JWT token.

Endpoint:

POST /api/auth/login

5. JWT Authentication  
Upon successful login, a JWT token is generated and stored inside an httpOnly cookie.
6. Logout Functionality  
A logout endpoint was implemented to clear the authentication cookie.

Endpoint:

POST /api/auth/logout

7. Current User Endpoint  
A protected endpoint was created to restore user sessions.

Endpoint:

GET /api/auth/me

8. Protected Routes  
React ProtectedRoute component was created to restrict access to authenticated users only.

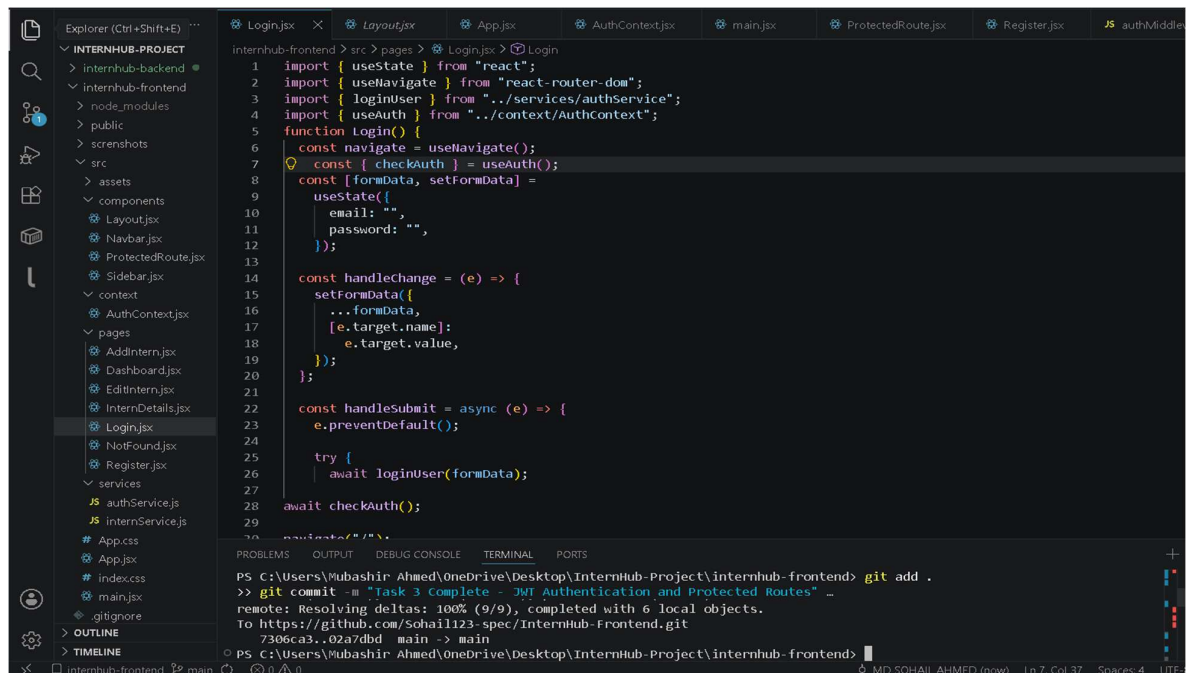
Protected Pages:

- Dashboard
  - Add Intern
  - Edit Intern
  - Intern Details
9. Session Persistence  
The application restores the logged-in user session using JWT cookies and the /api/auth/me endpoint.

Security Notes

1. Passwords are hashed using bcrypt.
2. JWT tokens are stored in httpOnly cookies.
3. Sensitive routes are protected using authentication middleware.
4. Unauthorized users receive 401 Unauthorized responses.
5. User sessions persist securely after browser refresh.
6. Tokens are not stored in localStorage.

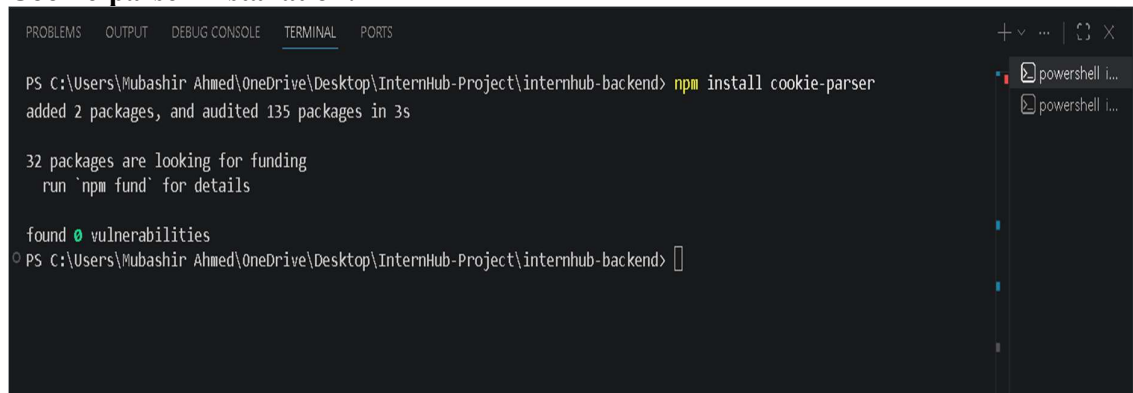
## Project Folder Structure:



## Description:

The frontend project follows a modular React architecture with separate folders for pages, components, services, and authentication management. This structure improves maintainability and scalability.

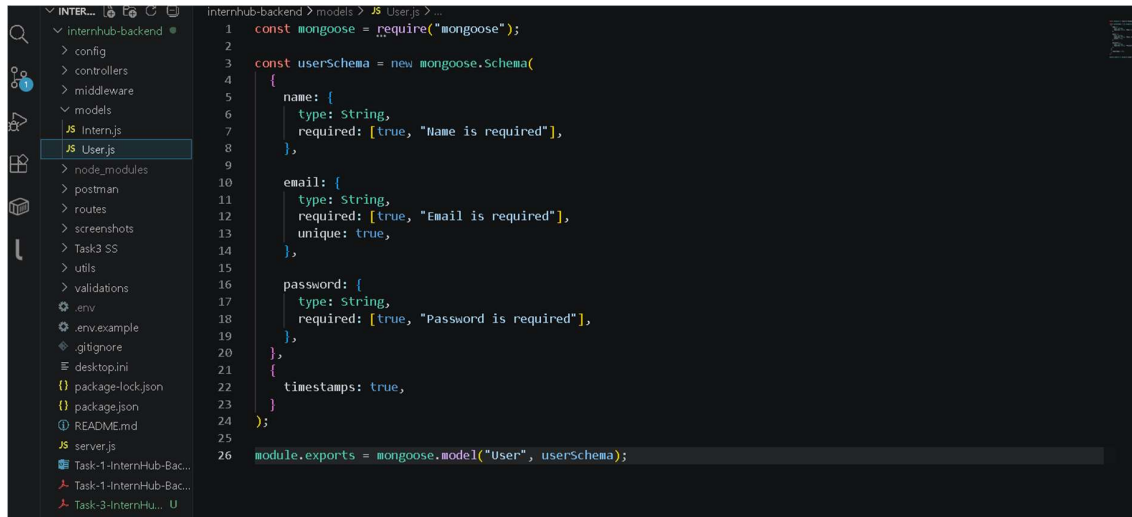
## Cookie-parser Installation:



## Description:

Required dependencies such as React Router DOM, Axios, JWT-related libraries, and authentication utilities were installed to support routing, API communication, and authentication functionality.

## User Model Creation:

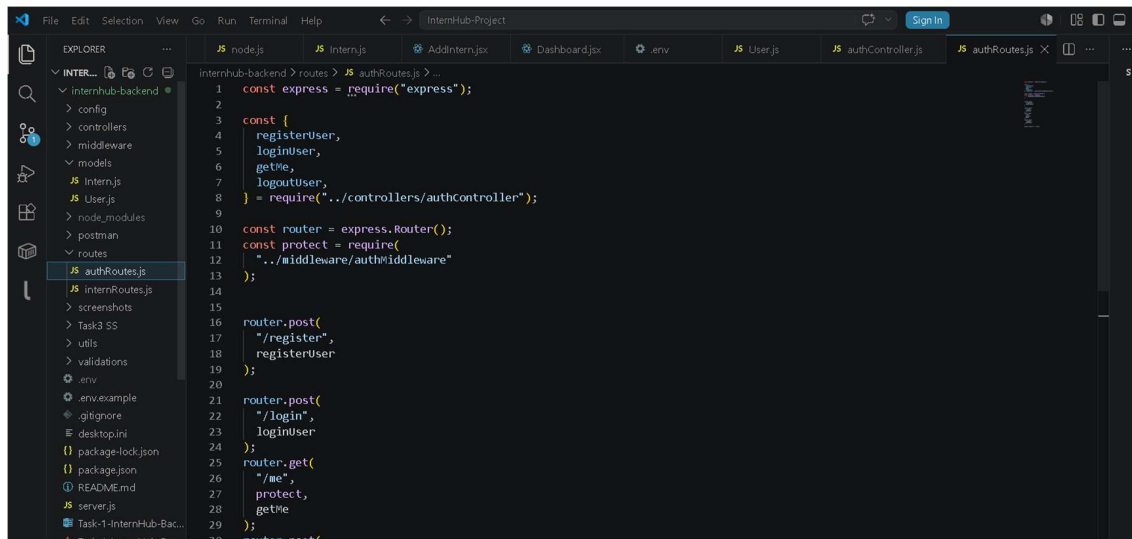


```
1 const mongoose = require("mongoose");
2
3 const userSchema = new mongoose.Schema(
4   {
5     name: {
6       type: String,
7       required: [true, "Name is required"],
8     },
9
10    email: {
11      type: String,
12      required: [true, "Email is required"],
13      unique: true,
14    },
15
16    password: {
17      type: String,
18      required: [true, "Password is required"],
19    },
20  },
21  {
22    timestamps: true,
23  }
24);
25
26 module.exports = mongoose.model("User", userSchema);
```

## Description:

A User schema was created using Mongoose to store user information including name, email, and hashed password. The schema serves as the foundation of the authentication system.

## Authentication Routes Setup:

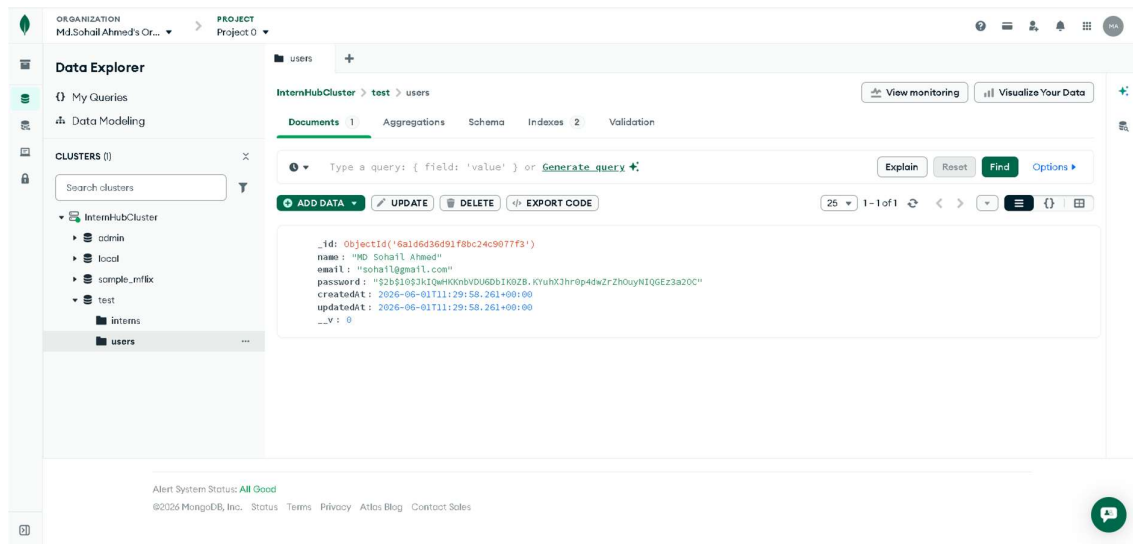


```
1 const express = require("express");
2
3 const {
4   registerUser,
5   loginUser,
6   getMe,
7   logoutUser,
8 } = require("../controllers/authController");
9
10 const router = express.Router();
11 const protect = require("../middleware/authMiddleware");
12
13 );
14
15
16 router.post(
17   "/register",
18   registerUser
19 );
20
21 router.post(
22   "/login",
23   loginUser
24 );
25
26 router.get(
27   "/me",
28   protect,
29   getMe
30 );
31
32 router.post(
```

## Description:

Authentication routes were implemented for user registration, login, logout, and current-user verification. These routes provide secure access control for the application.

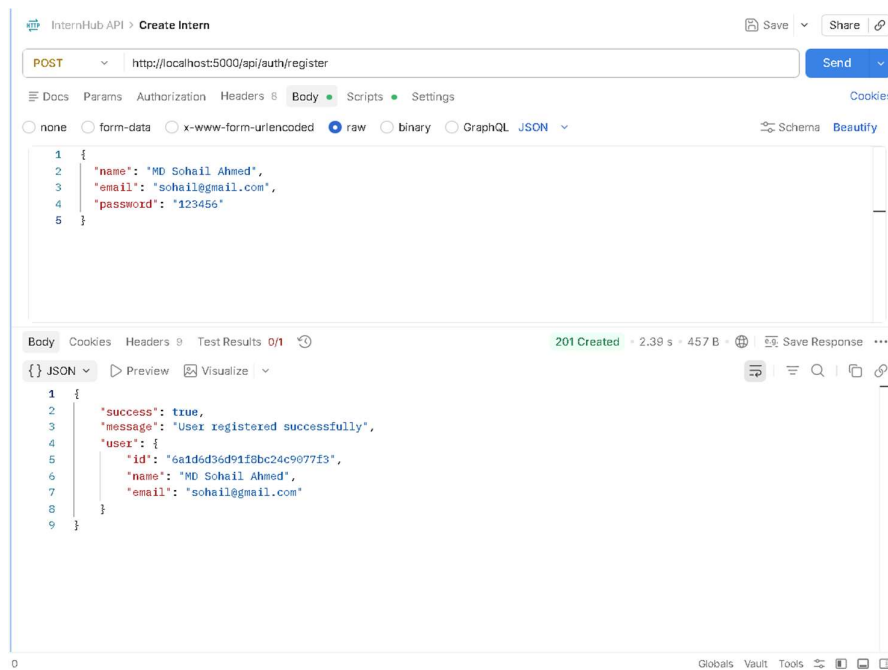
## Password Hashing Implementation:



## Description:

bcrypt was integrated to hash user passwords before storing them in MongoDB. This prevents plain-text password storage and improves security.

## User Registration API Testing:



## Description:

The registration endpoint was tested successfully using Postman. New user accounts were created and stored securely in the database.

## User Login API Testing:

The screenshot shows the Postman interface for testing the 'Create Intern' API endpoint. The request is a POST to 'http://localhost:5000/api/auth/login' with a JSON body containing email and password. The response is a 200 OK status with a JSON body indicating successful login and user details.

**Request:**

```
POST http://localhost:5000/api/auth/login
```

**Body (raw):**

```
{
  "email": "sohail@gmail.com",
  "password": "123456"
}
```

**Response (JSON):**

```
{
  "success": true,
  "message": "Login successful",
  "user": {
    "id": "6a1d6d36d91f8bc24c9077f3",
    "name": "MD Sohail Ahmed",
    "email": "sohail@gmail.com"
  }
}
```

## Description:

The login endpoint was tested successfully. JWT tokens were generated upon successful authentication and delivered through secure cookies.

## JWT Cookie Verification:

The screenshot shows the Chrome DevTools Cookies panel. A cookie named 'token' is listed for the domain 'localhost'. The cookie value is a long JWT token string. The cookie is marked as 'HttpOnly' and 'Secure'.

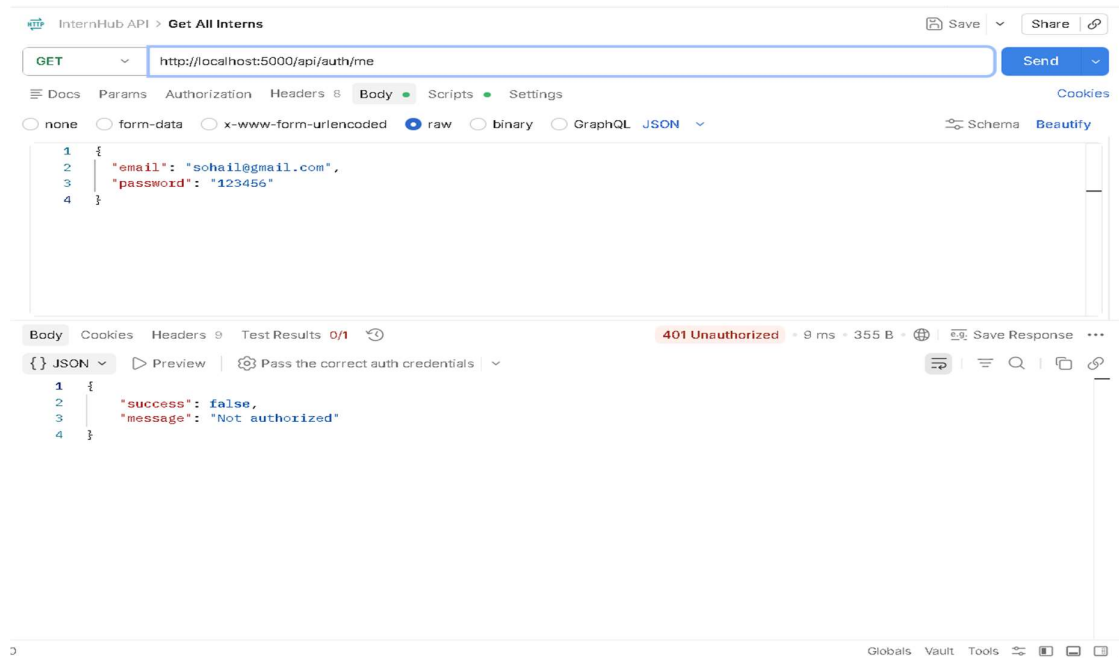
**Cookies Panel:**

- Domain: localhost
- Cookie Name: token
- Cookie Value:eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6IjZhMTd6d36d91f8bc24c9077f3IiwiaWF0Ij0yZDZkZjhiYzI0YzkwNzdmMyIsIm1hdCI6MTc4MDMxMzk3OCw1ZXhwIj0xNzg0TE4Nzc4fQ.kNUuPbw8KtHHY2fKX7mZSSc5\_0P-qIW0TPf8EBkKwhQ; Path=/; HttpOnly; Expires=Mon, 08 Jun 2026 11:39:38 GMT;

### Description:

Authentication cookies were verified through browser developer tools. The JWT token was successfully stored as an httpOnly cookie after login.

### Current User Authentication Check:



### Description:

The endpoint was tested to verify authentication enforcement. Unauthorized requests correctly returned a 401 Unauthorized response, confirming that protected routes are secured.

### User Registration Interface:

## Register

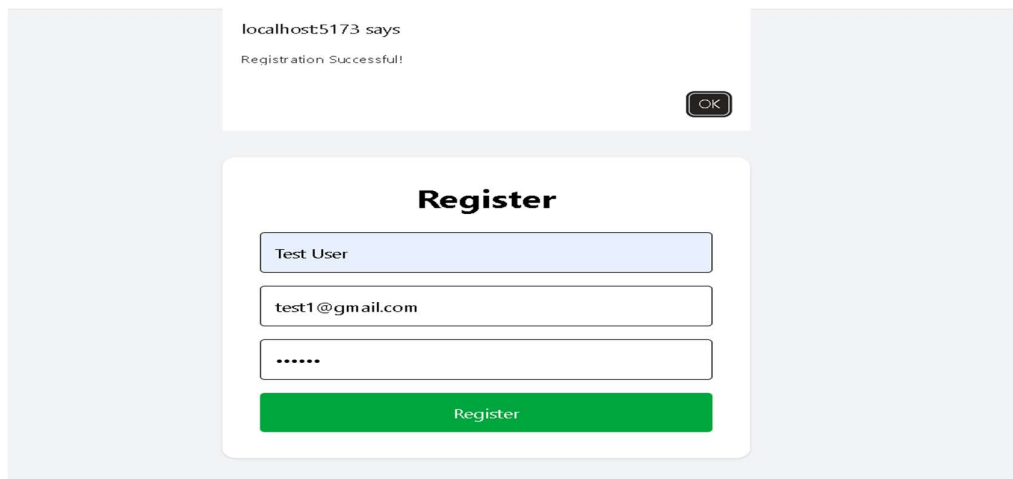
### Description:

The user registration page was developed using React functional components and controlled form inputs. New users can create an account by providing their name, email address, and password. The form data is validated and submitted to the backend authentication API.

### Key Features:

- React Hooks (useState)
- Form Validation
- API Integration using Axios
- User-Friendly Interface

### Successful User Registration:



The screenshot shows a web application interface. At the top, a notification box displays the text "localhost:5173 says" and "Registration Successful!" with an "OK" button. Below this is a "Register" form. The form contains three input fields: the first is labeled "Test User", the second is labeled "test1@gmail.com", and the third is a password field with six dots. A green "Register" button is at the bottom of the form.

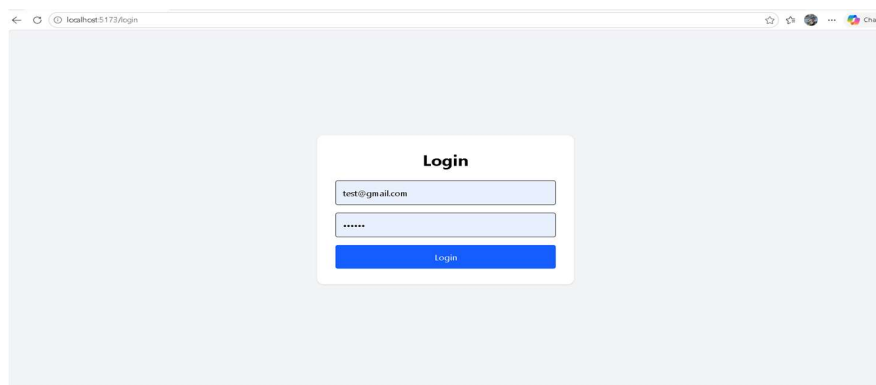
### Description:

After entering valid registration details, the user account is successfully created in MongoDB. Passwords are securely hashed using bcrypt before storage to ensure data security.

### Implementation Highlights:

- Password Hashing with bcrypt
- MongoDB User Storage
- Registration API Endpoint
- Secure User Creation Process

### Authentication & Route Protection Workflow:



The screenshot shows a web browser window with the address bar displaying "localhost:5173/login". The main content area features a "Login" form. The form has two input fields: the first is labeled "test@gmail.com" and the second is a password field with six dots. A blue "Login" button is at the bottom of the form.



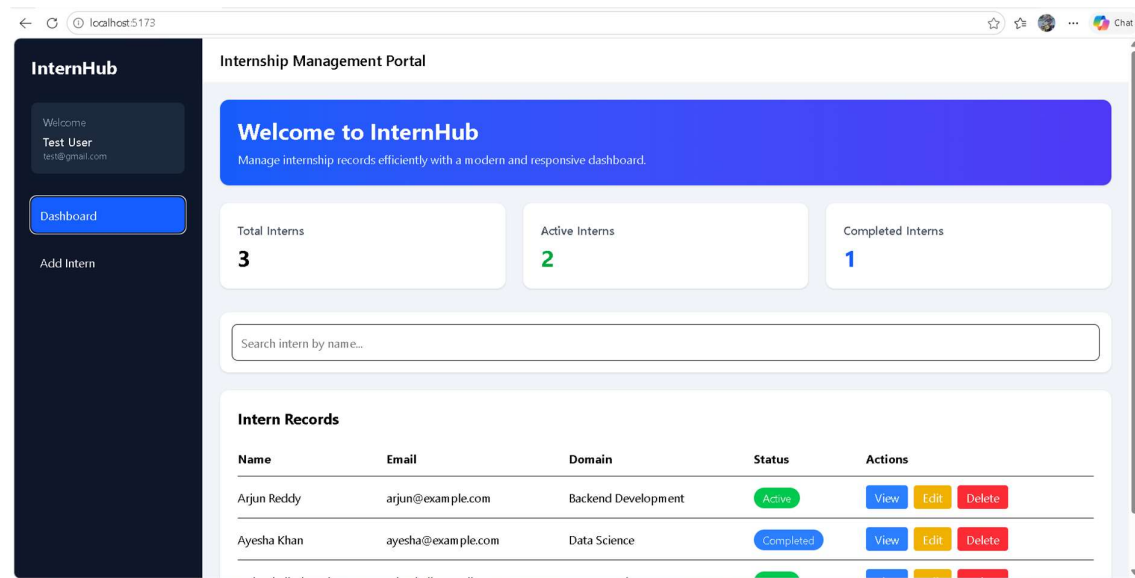
### Description:

Protected routes were implemented using React Router and a custom ProtectedRoute component. When an unauthenticated user attempts to access a protected page, the application automatically redirects the user to the login page.

### Security Benefits:

- Prevents Unauthorized Access
- Authentication Verification Before Page Access
- Improved Application Security
- User Session Validation

### Session Persistence After Refresh:



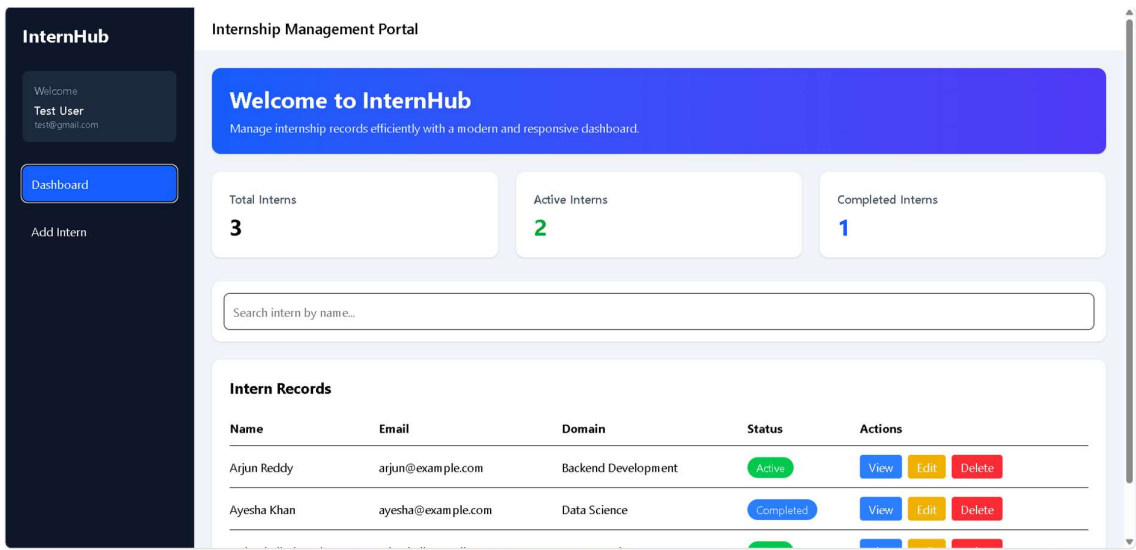
### Description:

The application successfully maintains the user session after browser refresh. Authentication state is restored using JWT stored in an httpOnly cookie and validated through the /api/auth/me endpoint.

### Key Features:

- Session Persistence
- JWT Authentication
- Automatic User Restoration
- Improved User Experience

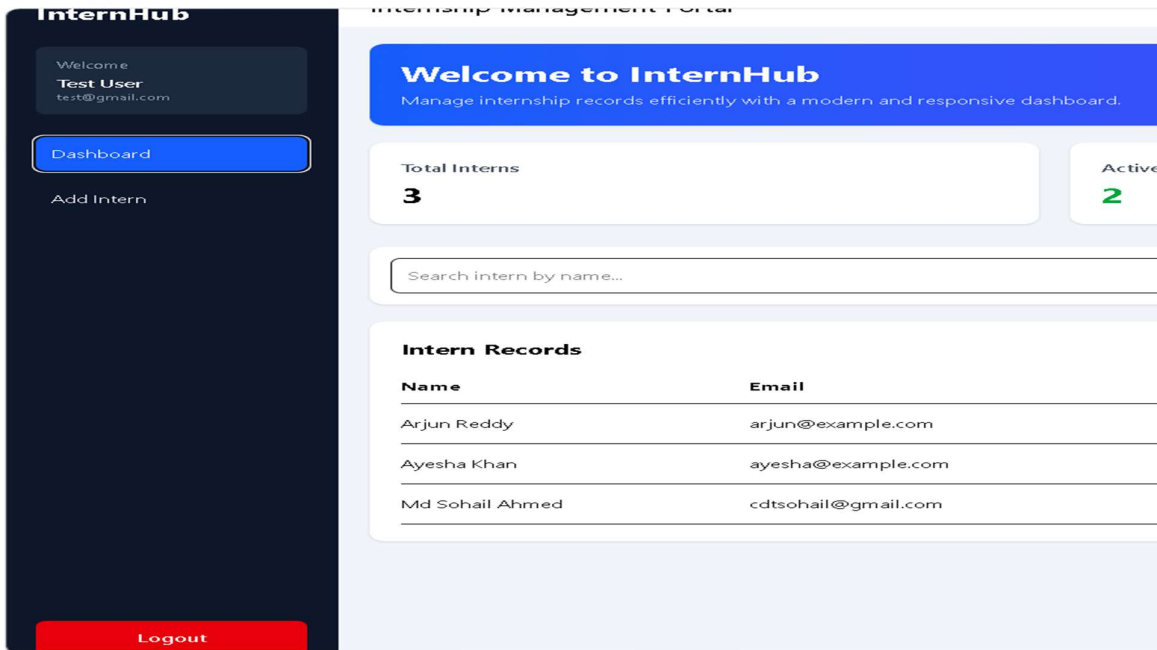
Authenticated Dashboard Access:



**Description:**  
After successful authentication, users gain access to the Internship Management Dashboard. The dashboard displays internship statistics, search functionality, and internship records fetched from protected backend APIs.

- Features Demonstrated:**
- Protected Dashboard Access
  - API Data Fetching
  - Internship Statistics
  - Internship Record Management

User Profile & Authentication State:



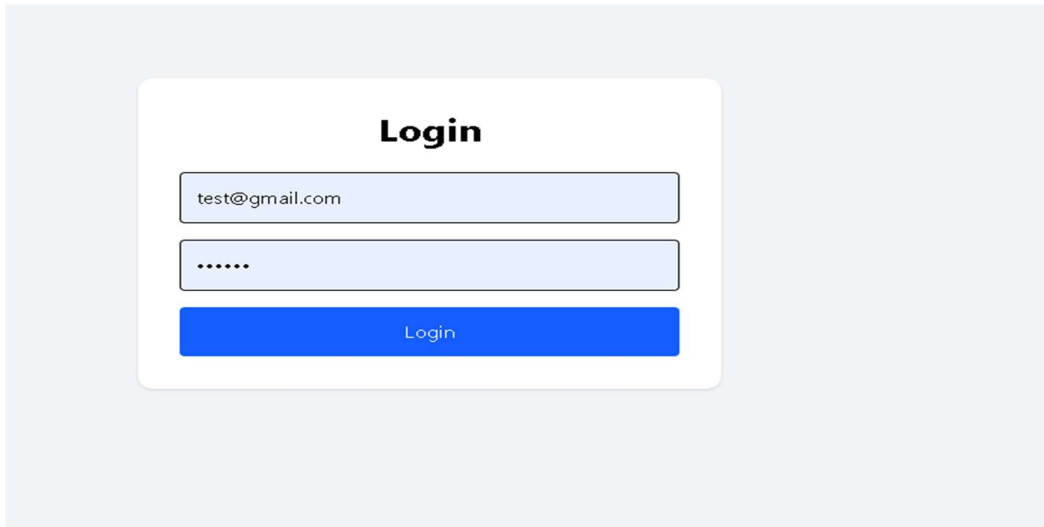
#### Description:

Authenticated user information is displayed within the application sidebar. The user profile section dynamically retrieves and displays the logged-in user's details using React Context API.

#### Features:

- User Profile Display
- React Context API
- Authentication State Management
- Personalized User Experience

#### Logout Functionality:



#### Description:

The logout feature securely removes the authentication cookie and clears the user session. After logout, protected pages become inaccessible and users are redirected to the login page.

#### Security Features:

- Secure Session Termination
- Cookie Removal
- Automatic Redirection
- Protected Route Enforcement

#### Security Measures & Outcome:

- Password Hashing using bcrypt
- JWT-Based Authentication
- httpOnly Cookie Storage
- Protected API Routes
- Protected React Routes
- Session Persistence
- Secure Logout Mechanism

#### Outcome:

A complete JWT-based authentication system was successfully implemented for the Internship Management Portal. The system provides secure user registration, login, logout, session persistence, route protection, and user authentication management while following modern web security practices.

## **Conclusion:**

In this task, a complete authentication system was successfully implemented for the InternHub application using JWT (JSON Web Token) and secure password hashing with bcrypt. User registration and login functionalities were developed, ensuring that passwords are never stored in plain text. JWT tokens were issued upon successful authentication and stored securely in HTTP-only cookies to improve application security.

Protected routes were implemented on both the backend and frontend, allowing only authenticated users to access sensitive pages and API endpoints. Session persistence was achieved through cookie-based authentication, enabling users to remain logged in even after refreshing the application. Logout functionality was also implemented to securely terminate user sessions.

This task significantly enhanced the security and reliability of the InternHub platform by introducing industry-standard authentication and authorization practices commonly used in modern web applications.

## **Outcome:**

- ✓ Secure user registration with bcrypt password hashing
- ✓ JWT-based authentication implemented
- ✓ HTTP-only cookie-based session management
- ✓ Protected backend API endpoints
- ✓ Protected frontend routes using React
- ✓ Session persistence after page refresh
- ✓ Secure logout functionality
- ✓ Improved application security following industry best practices